MITRC

B. TECH. CSE- Sem-4
4CS4-05: Introduction to Python Programming

MITRC
MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Chakravarty Sharma

Subject & Subject Code: Introduction to Python Programming & 4CS4-05

Semester: IV [CSE] **Session:** 2022-23

Unit No.: 1 **Date of Submission:** 28/02/2023

Lecture No.	Topics Covered	Page No.	Total Page
1	Introduction Python : Why Python? Python libraries & features	2- 4	3
2	Data types, variables, identifiers , key-words & expressions	5 - 8	4
3	Python Data Types, Comments & Indentation	9-11	3
4	Python - Variable names ,rules & multi words variable names- several techniques ,assign multiple values with examples	12-15	4
5	Multi words variable names- several techniques ,assign multiple values with examples	16-18	3
6	Python : numbers, strings & multiline strings with examples	19-22	4
7	String related built-in functions with examples	23-28	6
8	Python operators & practice of programs	29-31	3

References:

1. Programming Python ny Mark Lutz, O`Reilly
2. Learning Python, 3rd Edition by Mark Lutz, O`Reilly

21/2/2023

Faculty **HOD** **Dean Academic**

Page # 1/31

Lecture No. 01

Python Introduction

Python is a dynamic, interpreted (byte code-compiled) language. There are no type declarations of variables, parameters, functions, or methods in source code. This makes the code short and flexible, and you lose the compile-time type checking of the source code. Python tracks the types of all values at runtime and flags code that does not make sense as it runs.

What is Python?

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.

- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.
- Python Syntax compared to other programming languages
- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Python Indentation

Indentation refers to the spaces at the beginning of a code line.

Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to indicate a block of code.

Example

```
if 5 > 2:
print("Five is greater than two!")
```

Python will give you an error if you skip the indentation:

Example

Syntax Error:

```
if 5 > 2:
print("Five is greater than two!")
```

The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

Example

```
a=4
b=5
if a>b:
    h=a
else:
    h=b
print"Highest Value : ",a)
```

Python Comments

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

Creating a Comment

Comments starts with a #, and Python will ignore them:

Example

```
# Find highest value of any two values
```

Lecture No. 02

The Python Standard Library

What is the Python Libraries?

We know that a module is a file with some Python code, and a package is a directory for sub packages and modules. But the line between a package and a Python library is quite blurred.

A Python library is a reusable chunk of code that you may want to include in your programs/ projects.

Python Standard Library

The Python Standard Library is a collection of exact syntax, token, and semantics of Python. It comes bundled with core Python distribution. We mentioned this when we began with an introduction.

But in addition to this library, you can also access a growing collection of several thousand components from the Python Package Index (PyPI)..

Important Python Libraries

These are also the Python libraries for Data Science.

1. **Matplotlib**

Matplotlib helps with data analyzing, and is a numerical plotting library. We talked about it in Python for Data Science.

- Python Libraries - Python matplotlib
- Python Libraries Tutorial- matplotlib

2. Pandas

Pandas is a must for data-science.

It provides fast, expressive, and flexible data structures to easily (and intuitively) work with structured (tabular, multidimensional, potentially heterogeneous) and time-series data.

- Python Pandas
- Python Libraries Tutorial – Pandas

3. **Requests**

Requests is a Python Library that lets you send HTTP/1.1 requests, add headers, form data, multipart files, and parameters with simple Python dictionaries.

It also lets you access the response data in the same way.

- Python Requests Library
- Python Libraries Tutorial- Requests

4. NumPy

It has advanced math functions and a rudimentary scientific computing package.

- Python NumPy
- Python Libraries Tutorial – NumPy

5**. SQLAlchemy**

- SQLAIchemy Overview
- Python Libraries Tutorial – SQLAIchemy Overview

SQLAlchemy is a library with well-known enterprise-level patterns.

It was designed for efficient and high-performing database-access.

6. BeautifulSoup

It may be a bit slow, BeautifulSoup has an excellent XML- and HTML- parsing library for beginners.

- BeautifulSoup
- Python Libraries Tutorial – BeautifulSoup

7. Pyglet

Pyglet is an excellent choice for an object-oriented programming interface in developing games.

In fact, it also finds use in developing other visually-rich applications for Mac OS X, Windows, and Linux.

Python Pyglet

Python Libraries Tutorial – Pyglet

8. SciPy

Next up is SciPy, one of the libraries we have been talking so much about. It has a number of user-friendly and efficient numerical routines.

These include routines for optimization and numerical integration.

- Python SciPy
- Python Libraries Tutorial- SciPy

9. Scrapy

If your motive is fast, high-level screen scraping and web crawling, go for Scrapy.

You can use it for purposes from data mining to monitoring and automated testing.

- Python Scrapy
- Python Libraries Tutorial- Scrapy

10. PyGame

PyGame provides an extremely easy interface to the Simple Directmedia Library (SDL) platform-independent graphic, audio, and input libraries.

- Python PyGame
- Python Libraries Tutorial – PyGame

11. Python Twisted

An event-driven networking engine, Twisted is written in Python, and licensed under the open-source MIT license.

- Python Library - Python Twisted

- Python Libraries Tutorial – Twisted

12. Pillow

Pillow is a friendly fork of PIL (Python Imaging Library), but is more user-friendly.

- Python Library - Python Pillow
- Python Libraries Tutorial- Pillow

13. iPython

iPython Python Library has an architecture that facilitates parallel and distributed computing.

With it, you can develop, execute, debug, and monitor parallel applications.

- Python Library - iPython
- Python Library – iPython

14. Nose

Nose delivers an alternate test discovery and running process for unittest. This intends to mimic py.test's behavior as much as it can.

- Python Library - Python Nose
- Python Nose Library

15. Flask

A web framework, Flask is built with a small core and many extensions.

- Python Library - Python Flask
- Python Flask Library

16. SymPy

It is an open-source library for symbolic math.

With very simple and comprehensible code that is easily extensible, SymPy is a full-fledged Computer Algebra System (CAS).

It is written in Python, and hence does not need external libraries.

- Python Library - Python SymPy
- Python SymPy Library

Lecture No. 03

Built-in Data Types

In programming, data type is an important concept.

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

DATA TYPES

Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType

Getting the Data Type

You can get the data type of any object by using the type() function:

Example

Print the data type of the variable x:

```
x = 5
print(type(x))
```

Python Numbers

There are three numeric types in Python:

- int
- float
- complex

Variables of numeric types are created when you assign a value to them:

Example

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex
```

To verify the type of any object in Python, use the type() function:

Example

```
print(type(x))
print(type(y))
print(type(z))
```

Int

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

Example

Integers:

```
x = 1
y = 35656222554887711
z = -3255522

print(type(x))
print(type(y))
print(type(z))
```

Float

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

Example

Floats:

x = 1.10

y = 1.0

z = -35.59

print(type(x))

print(type(y))

print(type(z))

Float can also be scientific numbers with an "e" to indicate the power of 10.

Example

Floats:

x = 35e3

y = 12E4

z = -87.7e100

print(type(x))

print(type(y))

print(type(z))

Lecture No. 04

Python - Variable Names

Variables do not need to be declared with any particular type, and can even change type after they have been set.

Example

```
x = 4       # x is of type int
x = "ALWAR" # x is now of type str
print(x)
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

Example

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

Python - Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Example

Legal variable names:

```
myvar = "Umesh"
my_var = "Umesh"
_my_var = "Umesh"
myVar = "Umesh"
MYVAR = "Umesh"
myvar2 = "Umesh"
```

Example

Illegal variable names:

```
2myvar = "Umesh"
my-var = "Umesh"
my var = "Umesh"
```

Note : variable names are case-sensitive

Multi Words Variable Names
Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable:

Camel Case
Each word, except the first, starts with a capital letter:

myVariableName = "Kamal"

Pascal Case
Each word starts with a capital letter:

MyVariableName = "Kamal"

Snake Case
Each word is separated by an underscore character:

my_variable_name = "Kamal"

Python Variables - Assign Multiple Values

Many Values to Multiple Variables
Python allows you to assign values to multiple variables in one line:

Example
x, y, z = "Alwar", "Kota", "Jaipur"
OR
x=”Alwar”
y=”Kota”
z=”Jaipur

print(x)
print(y)
print(z)

Note: Make sure the number of variables matches the number of values, or else you will get an error.

a,b,c=34,67 # Error

One Value to Multiple Variables
And you can assign the same value to multiple variables in one line:

Example

```
x = y = z = "Alwar"
print(x)
print(y)
print(z)
```

Unpack a Collection
If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking.

Example
Unpack a list:

```
city = ["KOTA", "ALWAR", "JAIPUR"]
x, y, z = city
print(x)
print(y)
print(z)
```

Python - Output Variables

Output Variables
The Python print() function is often used to output variables.

Example

```
x = "MITRC"
print(x)
```

In the print() function, you output multiple variables, separated by a comma:

Example

```
x = "MITRC"
y = "Alwar"
print(x, y)
```

You can also use the + operator to output multiple variables:

Example

```
x = "MITRC"
y = "Alwar"
print(x + y)
```

For numbers, the + character works as a mathematical operator:

Example

```
x = 5
y = 10
print(x + y)
```

In the print() function, when you try to combine a string and a number with the + operator, Python will give you an error:

Example
```
x = 5
y = "MITRC"
print(x + y)
```

The best way to output multiple variables in the print() function is to separate them with commas, which even support different data types:

Example
```
x = 5
y = "MITRC"
print(x, y)
```

Lecture No. 05

Multi Words Variable Names

Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable:

Camel Case
Each word, except the first, starts with a capital letter:

myVariableName = "Kamal"

Pascal Case
Each word starts with a capital letter:

MyVariableName = "Kamal"

Snake Case
Each word is separated by an underscore character:

my_variable_name = "Kamal"

Python Variables - Assign Multiple Values

Many Values to Multiple Variables
Python allows you to assign values to multiple variables in one line:

Example

```
x, y, z = "Alwar", "Kota", "Jaipur"
```

OR

```
x=”Alwar”
y=”Kota”
z=”Jaipur

print(x)
print(y)
print(z)
```

Note: Make sure the number of variables matches the number of values, or else you will get an error.

```
a,b,c=34,67  # Error
```

One Value to Multiple Variables
And you can assign the same value to multiple variables in one line:

Example
```
x = y = z = "Alwar"
print(x)
print(y)
print(z)
```

Unpack a Collection
If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking.

Example
Unpack a list:

```
city = ["KOTA", "ALWAR", "JAIPUR"]
x, y, z = city
print(x)
print(y)
print(z)
```

Python - Output Variables

Output Variables
The Python print() function is often used to output variables.

Example
```
x = "Computech Alwar"
print(x)
```
In the print() function, you output multiple variables, separated by a comma:

Example
```
x = "Computech"
y = "Alwar"
print(x, y)
```

You can also use the + operator to output multiple variables:

Example
```
x = "Computech"
y = "Alwar"
print(x + y)
```

For numbers, the + character works as a mathematical operator:

Example

```
x = 5
y = 10
print(x + y)
```

In the print() function, when you try to combine a string and a number with the + operator, Python will give you an error:

Example

```
x = 5
y = "Kamal"
print(x + y)
```

The best way to output multiple variables in the print() function is to separate them with commas, which even support different data types:

Example

```
x = 5
y = "Kamal"
print(x, y)
```

Lecture No. 06

Python Strings

Strings in python are surrounded by either single quotation marks, or double quotation marks.

'hello' is the same as "hello".

You can display a string literal with the print() function:

Example

```
print("Hello")
print('Hello')
```

Assign String to a Variable

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Example

```
a = "Hello"
print(a)
```

Multiline Strings

You can assign a multiline string to a variable by using **three quotes:**

Example

You can use three double quotes:

```
a = """Computer is an electronic device,
which capable to perform mathematical,
commercial & scientific probles ."""
print(a)
```

Or three single quotes:

Example

```
a = '''Computer is an electronic device,
which capable to perform mathematical,
commercial & scientific probles .'''
print(a)
```

Slicing Strings

You can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

Example
Get the characters from position 2 to position 5 (not included):

```
b = "COMPUTECH”
print(b[3:6])
```

Note: The first character has index 0.

Slice From the Start
By leaving out the start index, the range will start at the first character:

Example
Get the characters from the start to position 5 (not included):

```
b = "COMPUTECH”
print(b[:5])
```

Modify Strings
Python has a set of built-in methods that you can use on strings.

Upper Case
Example
The upper() method returns the string in upper case:

```
a = "computech”
print(a.upper())
```
OUTPUT
COMPUTECH

Lower Case
Example
The lower() method returns the string in lower case:

```
a = "COMPUTECH”
print(a.lower())
```

Remove Whitespace
Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

Example
The strip() method removes any whitespace from the beginning or the end:

```
a = "     COMPUTECH”
print(a.strip()) # returns "COMPUTECH”
```

String Concatenation
To concatenate, or combine, two strings you can use the + operator.

Example
Merge variable a with variable b into variable c:

```
a = "COMPUTECH”
b = "ALWAR"
c = a + b
print(c)
```

Example
To add a space between them, add a " ":

```
a= "COMPUTECH”
b = "ALWAR"

c = a + " " + b
print(c)
```

String Format

In the Python Variables chapter, we cannot combine strings and numbers like this:

Example

```
age = 36
txt = "My name is Kunal, I am " + age  (ERROR)
print(txt)
```

But we can combine strings and numbers by using the format() method!

The **format() method** takes the passed arguments, formats them, and places them in the string where the placeholders {} are:

Example
Use the format() method to insert numbers into strings:

```
age = 36
txt = "My name is Kunal, and I am {}"
print(txt.format(age))
```

The format() method takes unlimited number of arguments, and are placed into the respective placeholders:

Example

```
q = 2
ino =1101
rate = 500.50
myorder = "I want {} pieces of item {} for {} Rupees."
print(myorder.format(q, ino, rate))
```

You can use index numbers {0} to be sure the arguments are placed in the correct placeholders:

```
q = 2
ino =1101
rate = 500.50
myorder = "I want to pay {2} Rupees  for {0} pieces of item {1}."
print(myorder.format(q, ino, rate))
```

Lecture No. 07

Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert.

An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

Example
You will get an error if you use double quotes inside a string that is surrounded by double quotes:

txt = "We are the so-called "Vikings" from the north."
To fix this problem, use the escape character \":

Example
The escape character allows you to use double quotes when you normally would not be allowed:

txt = "We are the so-called \"Vikings\" from the north."
Escape Characters
Other escape characters used in Python:

Code	Result	Try it
\'	Single Quote	
\\	Backslash	
\n	New Line	
\r	Carriage Return	
\t	Tab	
\b	Backspace	
\f	Form Feed	
\ooo	Octal value	
\xhh	Hex value	

```
txt = "Hello\nWorld!"
print(txt)
```
```
Hello
World!
```

String Methods

Python has a set of built-in methods that you can use on strings.

Note: All string methods returns new values. They do not change the original string.

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title
isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table to be used in translations
partition()	Returns a tuple where the string is parted into three parts
replace()	Returns a string where a specified value is replaced with a specified value
rfind()	Searches the string for a specified value and returns the last position of where it was found
rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Fills the string with a specified number of 0 values at the beginning

Examples

```
txt = "computech, alwar, rajasthan."
x = txt.capitalize()
print (x)
```

OUTPUT
Computech, Alwar, Rajasthan

```
txt = " Computech, Alwar, Rajasthan"

x = txt.casefold()

print(x)
```

OUTPUT
computech, alwar, rajasthan

```
txt = "COMPUTECH"

x = txt.center(20)
```

Print the word "COMPUTECH", taking up the space of 20 characters, with "COMPUTECH" in the middle:
print(x)
OUTPUT
COMPUTECH

String count() Method

Example
Return the number of times the value "apple" appears in the string:

```
txt = "I love apples, apple are my favorite fruit"

x = txt.count("apple")

print(x)
```
OUTPUT
2

The count() method returns the number of times a specified value appears in the string.

SYNTAX
string.count(value, start, end)

Parameter Values

Parameter	Description
value	Required. A String. The string to value to search for
start	Optional. An Integer. The position to start the search. Default is 0
end	Optional. An Integer. The position to end the search. Default is the end of the string

```
txt = "I love apples, apple are my favorite fruit"

x = txt.count("apple", 10, 24)

print(x)
```

OUTPUT
1

isalpha() Method

Check if all the characters in the text are letters
The isalpha() method returns True if all the characters are alphabet letters (a-z).

Example of characters that are not alphabet letters: (space)!#%&? etc.

```
txt = "CompanyX"

x = txt.isalpha()

print(x)
```

OUTPUT
True

isupper() Method

Check if all the characters in the text are in upper case:

```
txt = "COMPUTECH ,ALWAR”
x = txt.isupper()

print(x)
```

OUTPUT
TRUE

zfill() Method

Fill the string with zeros until it is 10 characters long.

SYNTAX
string.zfill(len)

Parameter	**Description**
len	Required. A number specifying the desired length of the string

Example
txt = "12"

x = txt.zfill(4)
print(x)
OUTPUT
0012

a = "COMPUTECH"
b = "ALWAR"
c = "RAJ."
print(a.zfill(10))
print(b.zfill(10))
print(c.zfill(10))

OUTPUT WILL BE
0COMPUTECH
00000ALWAR
000000RAJ.

Python Booleans

Booleans represent one of two values: **True or False.**

Example
print(10 > 9)
print(10 == 9)
print(10 < 9)

When you run a condition in an if statement, Python returns True or False:

Example
Print a message based on whether the condition is True or False:

a = 200
b = 33

```
if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```

Evaluate Values and Variables
The bool() function allows you to evaluate any value, and give you True or False in return,

Example
Evaluate a string and a number:

```
print(bool("Hello"))
print(bool(15))
```

Example
Evaluate two variables:

```
x = "Hello"
y = 15

print(bool(x))
print(bool(y))
```

Lecture No. 08

Python Operators

Operators are used to perform operations on variables and values.

In the example below, we use the + operator to add together two values:

Example
print(10 + 5)

Python divides the operators in the following groups:

1. Arithmetic operators
2. Assignment operators
3. Comparison operators
4. Logical operators
5. Identity operators
6. Membership operators
7. Bitwise operators
8.

Python Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Python Assignment Operators

Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3

//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Python Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example
x	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Python Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

Python Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Python Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Python Bitwise Operators

Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off